

Kubernetes-05

Today's topics coverage :

Namespace

- Logical separation inside one Kubernetes cluster
- Used to create environments (dev, staging, prod)
- Same resource names allowed in different namespaces
- Provides isolation between teams/projects
- Helps apply policies and quotas per team
- Default namespace exists if not specified

Requests & Limits

Requests

- Minimum CPU/RAM guaranteed to container
- Scheduler uses this to place pod on node
- Pod will always get at least this much resource

Limits

- Maximum CPU/RAM container can use
- Prevents container from using entire node

When exceeded

- CPU > limit → throttled (slow)
- Memory > limit → pod killed (OOMKilled)

Requests = reservation

Limits = restriction

ResourceQuota

- Applied at **namespace level**
- Controls total resource usage of a team
- Restricts number of objects (pods, services, PVC, etc.)
- Prevents one team from consuming full cluster
- If quota exceeded → resource creation fails
- Used by admins in multi-team clusters

LimitRange

- Applied at **namespace level**
- Sets rules for each container/pod inside namespace
- Defines minimum resource allowed
- Defines maximum resource allowed
- Assigns default resources if developer doesn't specify
- Prevents creating very small or very large pods
- Works together with ResourceQuota

What it can control

- min CPU / memory
- max CPU / memory
- default CPU / memory
- default requests
- limit-to-request ratio

Why needed

- Developers may forget to add resources

- Some may request too much resources
- Keeps cluster stable and balanced

Tasks:

1. Create a namespace `dev-environment` and apply a resource-based quota that restricts the number of pods to 3 and services to 2.
2. Create a pod in the `prod-environment` namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.
3. In the `staging-environment` namespace, set a `LimitRange` that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.
4. Create a pod and a `NodePort` service in the default namespace, then create another pod in the `test` namespace and communicate between them using `Service DNS`.
5. Apply a `LimitRange` with a max limit/request ratio of 2 for memory in the `performance-environment` namespace, and test by creating a pod with mismatched resource requests and limits.

TASK 1 : Create a namespace `dev-environment` and apply a resource-based quota that restricts the number of pods to 3 and services to 2.

Create a namespace `dev-environment` and apply `ResourceQuota`

- `pods = 3`
- `services = 2`

A **namespace** is like a separate workspace inside your Kubernetes cluster.

It allows you to **organize** and **isolate** resources (like Pods, Services, Deployments, etc.) so that different teams or environments (e.g., `dev`, `test`, `prod`) don't interfere with each other.

So here, we're creating a namespace named **`dev-environment`** to hold all our development-related resources.

Step 1 — Create namespace

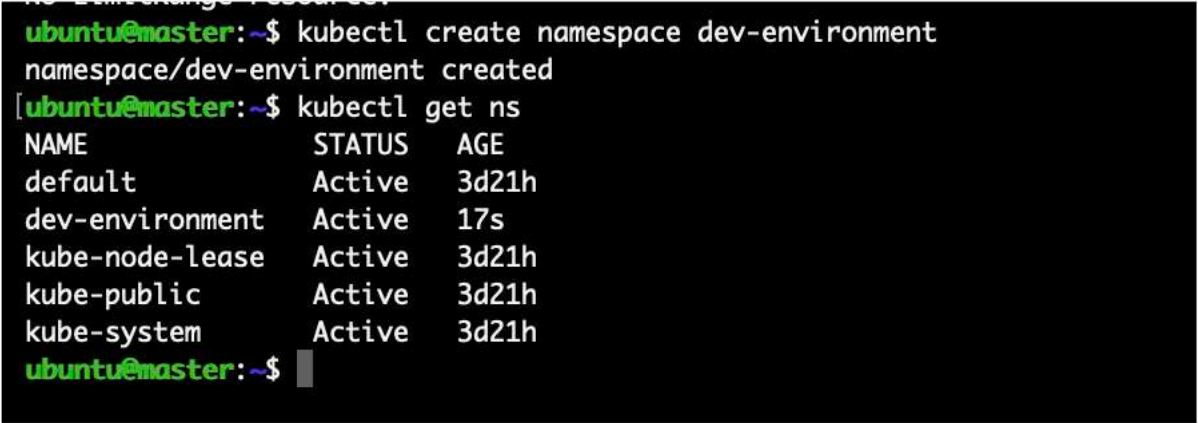
Run:

```
kubectl create namespace dev-environment
```

Check:

```
kubectl get ns
```

You should see dev-environment

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The first command is 'kubectl create namespace dev-environment', followed by the output 'namespace/dev-environment created'. The second command is 'kubectl get ns', followed by a table of namespaces. The table has three columns: NAME, STATUS, and AGE. The rows are: default (Active, 3d21h), dev-environment (Active, 17s), kube-node-lease (Active, 3d21h), kube-public (Active, 3d21h), and kube-system (Active, 3d21h). The prompt 'ubuntu@master:~\$' is shown again at the bottom.

```
ubuntu@master:~$ kubectl create namespace dev-environment
namespace/dev-environment created
ubuntu@master:~$ kubectl get ns
NAME                STATUS    AGE
default             Active    3d21h
dev-environment     Active    17s
kube-node-lease     Active    3d21h
kube-public         Active    3d21h
kube-system         Active    3d21h
ubuntu@master:~$
```

Step 2 — Create quota YAML

Create file:

```
vi dev-quota.yml
```

```
ubuntu@master:~$ vi dev-quota.yml
[ubuntu@master:~$ cat dev-quota.yml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev-environment
spec:
  hard:
    pods: "3"
    services: "2"

ubuntu@master:~$
```

Step 3 — Apply quota

kubectl apply -f dev-quota.yml
Verify:

kubectl describe quota dev-quota -n dev-environment

```
ubuntu@master:~$ kubectl apply -f dev-quota.yml
resourcequota/dev-quota created
ubuntu@master:~$ kubectl describe quota dev-quota -n dev-environment
Name:          dev-quota
Namespace:     dev-environment
Resource       Used  Hard
-----
pods           0    3
services       0    2
ubuntu@master:~$
```

Step 4 — Test

Create pods:

```
kubectl run pod1 --image=nginx -n dev-environment
kubectl run pod2 --image=nginx -n dev-environment
kubectl run pod3 --image=nginx -n dev-environment
kubectl run pod4 --image=nginx -n dev-environment
```

4th pod must FAIL

—> I have used three pods now lets test with 4th pod will it create or not

```
Last login: Fri Feb 20 09:07:16 2026 from 49.43.225.65
ubuntu@master:~$ kubectl run pod1 --image=nginx -n dev-environment
kubectl run pod2 --image=nginx -n dev-environment
[kubectl run pod3 --image=nginx -n dev-environment
pod/pod1 created
pod/pod2 created
pod/pod3 created
ubuntu@master:~$ kubectl describe quota dev-quota -n dev-environment
Name:          dev-quota
Namespace:     dev-environment
Resource       Used  Hard
-----
pods           3    3
services       0    2
ubuntu@master:~$
```

Now we can see when you try to create a **fourth Pod**, you'll get an error like this:

```
ubuntu@master:~$ kubectl run pod4 --image=nginx -n dev-environment
Error from server (Forbidden): pods "pod4" is forbidden: exceeded quota: dev-quota, requested: pods=1, used: pods=3, limited: pods=3
ubuntu@master:~$
```

Kubernetes is enforcing:

Team cannot exceed allocated resources

Even if cluster has free CPU/RAM → still blocked.

TASK-2: Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.

Create a **pod in prod-environment namespace** with:

- Request → 0.2 CPU & 200Mi memory
- Limit → 0.5 CPU & 500Mi memory

Here you will understand **how Kubernetes guarantees and restricts resources**.

STEP 1 — Create Namespace

```
kubectl create namespace prod-environment
```

Why?

We separate real application (production) from dev testing.

```
Cluster
├── dev-environment
└── prod-environment ← real users app
```

Check:

```
kubectl get ns
```

```
ubuntu@master:~$ kubectl create namespace prod-environment
namespace/prod-environment created
ubuntu@master:~$ kubectl get ns
NAME                STATUS    AGE
default             Active   3d21h
dev-environment     Active   15m
kube-node-lease     Active   3d21h
kube-public         Active   3d21h
kube-system         Active   3d21h
prod-environment    Active   19s
ubuntu@master:~$
```

STEP 2 — Understand Requests vs Limits (before writing YAML)

Think like this:

Resource	Meaning
Request	minimum guaranteed
Limit	maximum allowed

So we tell Kubernetes scheduler:

“My app needs at least 0.2 CPU to run properly
but never allow it to cross 0.5 CPU”

Important behavior

Condition	Result
Node doesn't have request CPU	pod NOT scheduled
CPU crosses limit	throttled (slow)
Memory crosses limit	pod killed (OOMKilled)

STEP 3 — Create Pod YAML

Open file:

```
vi prod-pod.yml
```



```
prod-environment Active 19s
ubuntu@master:~$ vi prod-pod.yml
[ubuntu@master:~$ cat prod-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod
  namespace: prod-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        cpu: "200m"
        memory: "200Mi"
      limits:
        cpu: "500m"
        memory: "500Mi"

ubuntu@master:~$ █
```

Understand YAML

Field	Meaning
requests	guaranteed resources
limits	maximum usage
200m cpu	0.2 core
500m cpu	half CPU core

STEP 4 — Apply Pod

```
kubectl apply -f prod-pod.yml
```

STEP 5 — Verify

```
kubectl describe pod prod-pod -n prod-environment
```

```
ubuntu@master:~$ kubectl apply -f prod-pod.yml  
pod/prod-pod created
```

```
ubuntu@master:~$ kubectl describe pod prod-pod -n prod-environment  
Name: prod-pod  
Namespace: prod-environment  
Priority: 0  
Service Account: default  
Node: worker1/172.31.9.88  
Start Time: Fri, 20 Feb 2026 09:28:17 +0000  
Labels: <none>  
Annotations: cni.projectcalico.org/containerID: bc1ba04dfd1e6ff802af26ec87204b70e286d41077e777a27eb31fcf4d4570ce  
              cni.projectcalico.org/podIP: 192.168.235.183/32  
              cni.projectcalico.org/podIPs: 192.168.235.183/32  
Status: Running  
IP: 192.168.235.183  
IPs:  
  IP: 192.168.235.183  
Containers:  
  nginx:  
    Container ID: containerd://89ee2070653a0b74a21d6f73f8bf00406239c32caedeac9746c7c402302148a  
    Image: nginx  
    Image ID: docker.io/library/nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208  
    Port: <none>  
    Host Port: <none>  
    State: Running  
      Started: Fri, 20 Feb 2026 09:28:17 +0000  
    Ready: True  
    Restart Count: 0  
    Limits:  
      cpu: 500m  
      memory: 500Mi  
    Requests:  
      cpu: 200m  
      memory: 200Mi  
    Environment: <none>  
    Mounts:  
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-f2dtj (ro)  
Conditions:  
  Type Status  
  PodReadyToStartContainers True  
  Initialized True
```

What we proved

Scheduler now reserves CPU & RAM before running pod.

So Kubernetes now knows:

“This application needs guaranteed performance”

This is **mandatory** in **production clusters**.

TASK-3 — In the staging-environment namespace, set a LimitRange that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

LimitRange

Namespace: **staging-environment**

We will create rules:

- Default CPU = 300m
- Default Memory = 600Mi
- Min CPU = 100m
- Max CPU = 800m

Meaning:

Even if developer forgets resources → Kubernetes auto assigns
And nobody can request too small or too large CPU

STEP 1 — Create namespace

```
kubectl create namespace staging-environment
```

Check:

```
kubectl get ns
```

```
ubuntu@master:~$ kubectl create namespace staging-environment
namespace/staging-environment created
[ubuntu@master:~$ kubectl get ns
NAME                STATUS    AGE
default             Active    3d21h
dev-environment     Active    25m
kube-node-lease     Active    3d21h
kube-public         Active    3d21h
kube-system         Active    3d21h
prod-environment    Active    10m
staging-environment Active    14s
ubuntu@master:~$
```

STEP 2 — Why LimitRange?

Problem in companies:

Developer mistake	Result
No resources defined	Pod eats all node memory
Too small CPU	App crashes
Too big CPU	Other apps starve

So admin creates **rules per container** → LimitRange

ResourceQuota controls TOTAL

LimitRange controls EACH POD

STEP 3 — Create LimitRange YAML

Open file:

```
vi staging-limit.yml
```

```

ubuntu@master:~$ vi staging-limit.yml
[ubuntu@master:~$ cat staging-limit.yml
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limit
  namespace: staging-environment
spec:
  limits:
  - type: Container
    default:
      cpu: "300m"
      memory: "600Mi"
    defaultRequest:
      cpu: "200m"
      memory: "300Mi"
    min:
      cpu: "100m"
    max:
      cpu: "800m"

```

```

ubuntu@master:~$ █

```

Understand fields

Field	Meaning
default	auto assign limit if not provided
defaultRequest	auto assign request
min	cannot go below
max	cannot exceed

STEP 4 — Apply rule

```
kubectl apply -f staging-limit.yml
```

Verify:

```
kubectl describe limitrange -n staging-environment
```

```
[ubuntu@master:~]$ kubectl apply -f staging-limit.yml
limitrange/staging-limit created
ubuntu@master:~$ kubectl describe limitrange -n staging-environment
Name:          staging-limit
Namespace:     staging-environment
Type           Resource  Min  Max  Default Request  Default Limit  Max Limit/Request Ratio
-----
Container      cpu       100m 800m 200m              300m            -
Container      memory   -    -    300Mi            600Mi           -
ubuntu@master:~$
```

STEP 5 — TEST (important)

Now create pod WITHOUT resources:

```
kubectl run testpod --image=nginx -n staging-environment
```

Check assigned resources:

```
kubectl describe pod testpod -n staging-environment
```

You will see automatically:

```

ubuntu@master:~$ kubectl run testpod --image=nginx -n staging-environment
pod/testpod created
ubuntu@master:~$ kubectl describe pod testpod -n staging-environment
Name:          testpod
Namespace:     staging-environment
Priority:       0
Service Account: default
Node:          worker2/172.31.9.239
Start Time:    Fri, 20 Feb 2026 09:39:45 +0000
Labels:        run=testpod
Annotations:    cni.projectcalico.org/containerID: 3901001beef50d012c0aa402e4dad892e99f5feda9ac7d2fbf736321ec7db853
                cni.projectcalico.org/podIP: 192.168.189.119/32
                cni.projectcalico.org/podIPs: 192.168.189.119/32
                kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, memory request for container testpod; cpu, mem
Status:        Running
IP:            192.168.189.119
IPs:           IP: 192.168.189.119
Containers:
  testpod:
    Container ID:  containerd://b77f4dfb9c691cb1852a1f98bf726a249c8a04a290942df3704cb065a4b1eb18
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Fri, 20 Feb 2026 09:39:46 +0000
    Ready:         True
    Restart Count:  0
    Limits:
      cpu:          300m
      memory:       600Mi
    Requests:
      cpu:          200m
      memory:       300Mi
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2twp4 (ro)
Conditions:
  Type            Status

```

Kubernetes added values automatically ✓

What we proved

LimitRange forces developers to follow company policy.

Even if YAML empty → cluster safe.

TASK-4 :Create a pod and a NodePort service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.

Communication using Service DNS

(This is VERY real-time — apps talk to each other like frontend → backend → database)

We will:

1. Create **web server pod + NodePort service (default namespace)**
2. Create **client pod in test namespace**
3. Access it using **Kubernetes internal DNS**

STEP 1 — Create nginx pod YAML (default namespace)

Create file:

```
vi web-pod.yml
```



```
ubuntu@master:~$ vi web-pod.yml
[ubuntu@master:~$ cat web-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80

ubuntu@master:~$
```

Apply:

```
kubectl apply -f web-pod.yml
kubectl get pods
```

```
ubuntu@master:~$ kubectl apply -f web-pod.yml
pod/web-pod created
ubuntu@master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
web-pod   1/1     Running   0           8s
ubuntu@master:~$
```

STEP 2 — Create service YAML

Create file:

```
vi web-service.yml
```

```
web pod 1/1 Running 0s
ubuntu@master:~$ vi web-service.yml
ubuntu@master:~$ cat web-service.yml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80

ubuntu@master:~$
```

Apply:

```
kubectl apply -f web-service.yml
kubectl get svc
```

```
ubuntu@master:~$ kubectl apply -f web-service.yml
service/web-service created
ubuntu@master:~$ kubectl get svc
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
kubernetes      ClusterIP   10.96.0.1     <none>         443/TCP        22h
web-service     NodePort    10.98.106.72  <none>         80:32584/TCP   14s
ubuntu@master:~$
```

What happened internally

Service checks labels:

Service selector: app=web

↓

Pod label: app=web

↓

Traffic connected

STEP 3 — Create test namespace

```
kubectl create namespace test
```

```
ubuntu@master:~$ kubectl create namespace test
namespace/test created
ubuntu@master:~$
```

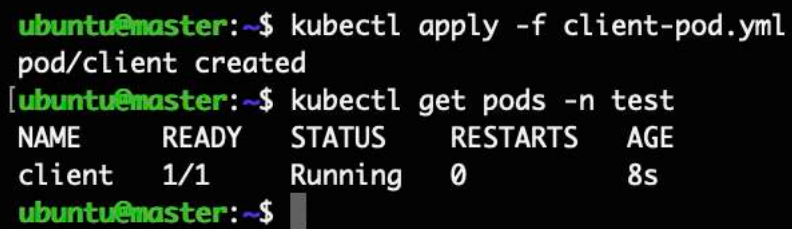
STEP 4 — Create client pod YAML

```
vi client-pod.yml
```

```
ubuntu@master:~$ kubectl create namespace test
namespace/test created
ubuntu@master:~$ vi client-pod.yml
[ubuntu@master:~$ cat client-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: client
  namespace: test
spec:
  containers:
  - name: busybox
    image: busybox
    command: ["sh", "-c", "sleep 3600"]
ubuntu@master:~$
```

Apply:

```
kubectl apply -f client-pod.yml  
kubectl get pods -n test
```

A terminal window with a black background and green text. It shows the execution of two kubectl commands. The first command, 'kubectl apply -f client-pod.yml', is followed by the output 'pod/client created'. The second command, 'kubectl get pods -n test', is followed by a table of pod information.

```
ubuntu@master:~$ kubectl apply -f client-pod.yml  
pod/client created  
[ubuntu@master:~$ kubectl get pods -n test  
NAME      READY   STATUS    RESTARTS   AGE  
client    1/1     Running   0           8s  
ubuntu@master:~$
```

STEP 5 — Enter client container

```
kubectl exec -it client -n test -- sh
```

STEP 6 — DNS communication test

Inside container:

```
wget -qO- web-service.default.svc.cluster.local
```

```
client 1/1 Running 0 8s
ubuntu@master:~$ kubectl exec -it client -n test -- sh
/ # wget -q0- web-service.default.svc.cluster.local

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ #
/ #
```

Expected output

You will see nginx HTML page:

Welcome to nginx!

TASK-5 — Apply a LimitRange with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.

LimitRange *limit-to-request ratio*

Namespace: **performance-environment**

We will enforce rule:

A container's **memory limit cannot be more than 2× its request**

First understand the concept

Example:

Request	Limit	Allowed?
200Mi	300Mi	✓ OK
200Mi	400Mi	✓ OK
200Mi	500Mi	✗ NOT OK

Because:

```
limit / request = ratio  
500 / 200 = 2.5 > allowed 2  
So Kubernetes blocks greedy containers.
```

This prevents developers from doing:

reserve very small memory but try to consume huge memory

STEP 1 — Create namespace

```
kubectl create namespace performance-environment
```

Check:

```
kubectl get ns
```

```
ubuntu@master:~$ kubectl create namespace performance-environment
namespace/performance-environment created
[ubuntu@master:~$ kubectl get ns
NAME                STATUS    AGE
default              Active    3d22h
dev-environment      Active    53m
kube-node-lease      Active    3d22h
kube-public          Active    3d22h
kube-system          Active    3d22h
performance-environment Active    7s
prod-environment     Active    38m
staging-environment  Active    28m
test                 Active    13m
ubuntu@master:~$
```

STEP 2 — Create LimitRange rule

Create file:

```
vi ratio-limit.yml
```

```
ubuntu@master:~$ vi ratio-limit.yml
[ubuntu@master:~$ cat ratio-limit.yml
apiVersion: v1
kind: LimitRange
metadata:
  name: memory-ratio-limit
  namespace: performance-environment
spec:
  limits:
  - type: Container
    maxLimitRequestRatio:
      memory: "2"
ubuntu@master:~$
```

Meaning

Kubernetes now enforces:

$\text{memory limit} \leq 2 \times \text{memory request}$

Apply:

```
kubectl apply -f ratio-limit.yml
kubectl describe limitrange -n performance-environment
```

```
ubuntu@master:~$ kubectl apply -f ratio-limit.yml
limitrange/memory-ratio-limit created
ubuntu@master:~$ kubectl describe limitrange -n performance-environment
Name:          memory-ratio-limit
Namespace:     performance-environment
Type
-----
Container      Resource  Min  Max  Default Request  Default Limit  Max Limit/Request Ratio
-----
Container      memory   -    -    -                 -              2
ubuntu@master:~$
```

STEP 3 — Create BAD pod (should fail)

Create file:

```
vi bad-pod.yml
```



```
container: memory
[ubuntu@master:~]$ vi bad-pod.yml
[ubuntu@master:~]$ cat bad-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: badpod
  namespace: performance-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "100Mi"
      limits:
        memory: "500Mi"

ubuntu@master:~$
```

Why this fails

limit = 500Mi
request = 100Mi

ratio = 5
allowed = 2

Apply:

kubectl apply -f bad-pod.yml

```
ubuntu@master:~$ kubectl apply -f bad-pod.yml
Error from server (Forbidden): error when creating "bad-pod.yml": pods "badpod" is forbidden: memory max limit to request ratio per Container is 2, but provided ratio is 5.000000
ubuntu@master:~$
```

Expected error

You will get something like:

Error from server (Forbidden):
maximum memory limit to request ratio per Container is 2

What we proved

LimitRange can also control **fairness**, not just min/max/default.

Prevents:

- cheating resource allocation
- node memory pressure
- noisy neighbor problem

Kubernetes Lab — Topics Covered

Namespace (environment separation)

- Created multiple namespaces (dev, prod, staging, test, performance)
- Learned same resource name can exist in different namespaces
- Used `-n <namespace>` to isolate applications

Purpose: separate teams & environments

ResourceQuota (team usage control)

- Limited namespace to **3 pods & 2 services**
- Tried creating 4th pod → blocked
- Understood quota controls **total usage**

Prevents one team from consuming whole cluster

Requests & Limits (container resource control)

- Created production pod with CPU & memory guarantees
- Understood scheduler reserves request resources
- Learned behavior:
 - CPU exceed → throttled
 - Memory exceed → OOMKilled

Used in real production apps

LimitRange (rules per container)

- Applied default CPU & memory automatically
- Set min & max CPU
- Pod created without resources → auto assigned values

Forces developers to follow company policy

Service & DNS Communication

- Created pod + service
- Accessed from another namespace
- Used internal DNS:

`service.namespace.svc.cluster.local`

- Tested using wget/curl inside pod

Learned: Pods never communicate using pod IP

LimitRange Ratio (advanced control)

- Enforced memory limit/request ratio = 2
- Created invalid pod → Kubernetes rejected

Prevents resource abuse (“reserve less, consume more”)

Big Concepts You Learned

Level	What it controls
Namespace	environment separation
ResourceQuota	total team usage
Requests/Limits	container resources
LimitRange	rules per container
Service DNS	pod communication
Ratio limit	fairness & stability

